Junior Java Backend fejlesztő V.

Botlik Béla

Interface, S.O.L.I.D, kollekciók

(A prezentációban felhasználásra kerültek diák a Ruander Oktatóközpont korábbi tananyagából. Szerzők: Béres Gábor, Czigléczky Gábor, Fehér Róbert, Kis Balázs)

Interface I.

- Az interface egy OO programozásban elfogadott elem, mely azonban az OO hierarchián kívül áll.
- Az interface lényegében egy formai megkötés, mely arra hivatott, hogy az az osztály, amelyik implementálja, kötelezően implementálnia kell az interface által megszabott alprogramot.
- Segítségével az adott OO hierarchia bővíthető különböző megkötésekkel, így elkerülhető a dupla öröklés, mely Java-ban nem engedélyezett.
- Interface = implementáció nélküli metódusok névvel ellátott halmaza.

Interface II.

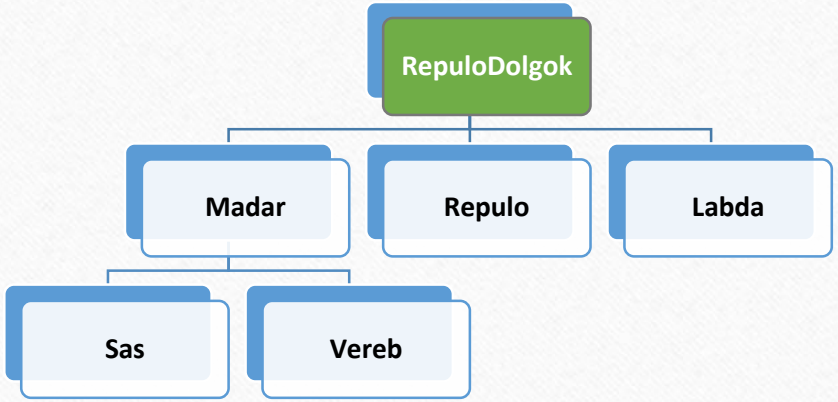
- Az interface és az absztrakt osztály, habár hasonló feladatokat látnak el, mégis hatalmas különbségek vannak köztük:
- Alapértelmezetten egy OO hierarchiában alapvetően a program szempontjából elsődlegesen fontos OO kialakítást megalkotjuk (absztrakt és nem absztrakt osztályok hierarchiájaként), majd a hierarchiába már nem beilleszthető funkcionalitásokat interface segítségével alakítjuk ki.

Interface vs Absztrakt osztály

Egyezések	Különbözőségek
• Mindkettő tud alprogram megkötéseket szabni az osztályra, ami absztrakt osztályból öröklődik, vagy az interface-t implementálja. • Egyikből sem hozható létre példány. • Mindkettő használható „gyűjtő típusnak” a boxing értelmében.	• Az interface nem OO hierarchiába tartozó elem. • Egy osztály interface-ekből korlátlan számút implementálhat. • Az interface láthatósági szintet nem állíthat.

Interface III.

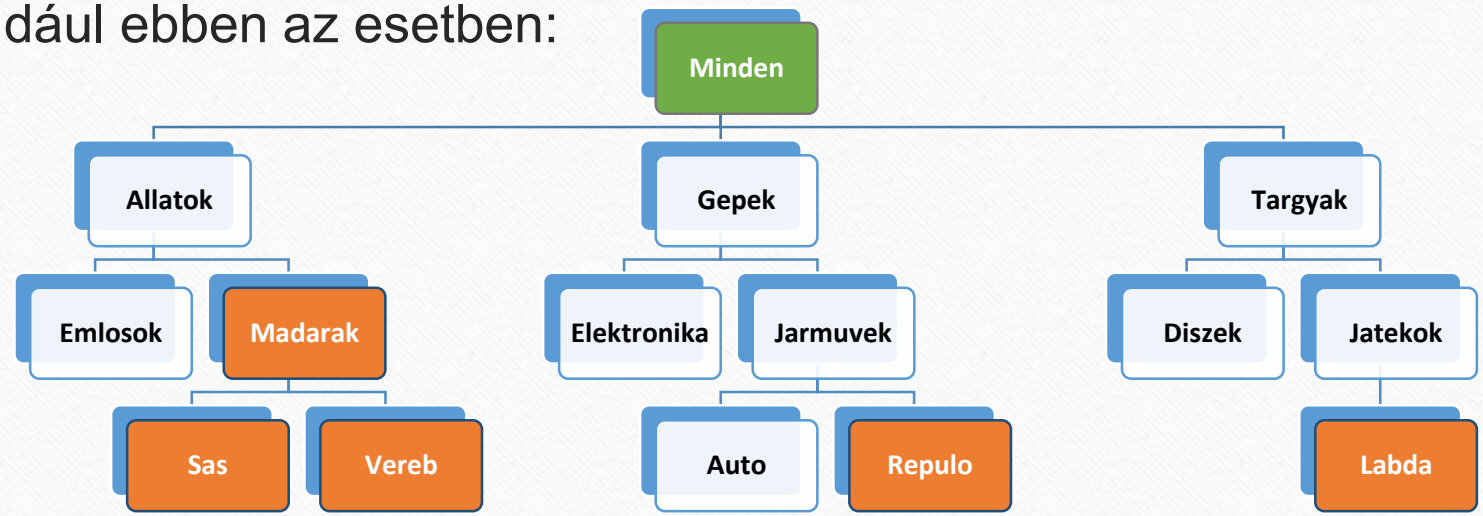
- Egy OO hierarchia:



- Ebben az esetben a különböző osztályok a RepuloDolgok közös ős alá vannak rendelve, azonban előfordulhat, hogy egy teljesen más hierarchiai felépítést kell kövessünk a program többi része miatt.

Interface IV.

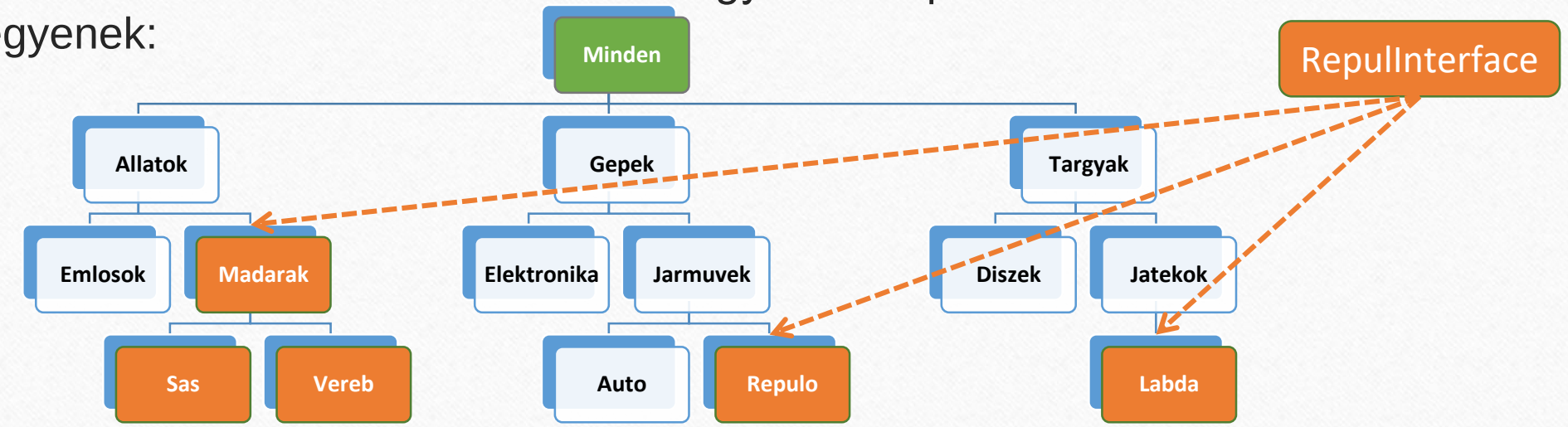
- Mint például ebben az esetben:



- Mint látható, ebben az esetben a naranccsal jelölt osztályok továbbra is tudnak repülni, azonban a legközelebbi közös ősük a Minden absztrakt osztály, ami azonban nem kényszerítheti rá minden leszármazottra a Repul() alprogramot.

Interface V.

- Így szükség van egy olyan „ősre”, mely megszabja továbbra is, hogy az adott osztályok tudnak repülni (repul()), illetve ezek az osztályok, ily módon polimorfizmus felhasználásával akár egy közös típusként is kezelhetők legyenek:



Interface VI.

- Megvalósítás:
  - interface <név> {
    // konstans deklarációk
    // metódus deklarációk
    }
- Példa:
  - interface RepulInterface() {
    int maxSebesseg = 300000;
    void repul();
    }

Interface VII.

- Java-ban számos beépített interface van már előre implementálva.
- Néhány fontosabb interface:
  - Comparable
    - Az osztály amelyik implementálja sorrend szerint összehasonlítható → compareTo()
  - Cloneable
    - Az osztály amelyik implementálja klónozható → clone()
  - Closeable
    - Az osztály amelyik implementálja felszabadíthatja a használt erőforrásokat → close()
  - Iterable
    - Az osztály amelyik implementálja iterálható, bejárható foreach szerkezettel.

Feladat I.

- Készítsünk alkalmazást, melyben egy cég kapcsolatait és szolgáltatásait tudjuk adminisztrálni.
- A kapcsolatokról tárolandó:
  - Név, Cím, Email, Telefon
- A szolgáltatásokról tárolandó:
  - Megnevezés, Ár
- Oldjuk meg, hogy mind a kapcsolatok, mind a szolgáltatások **egységesen** menthetők legyenek CSV formátumban.

S.O.L.I.D. I.

- A S.O.L.I.D. objektum orientált tervezési minta olyan ajánlásokat tesz, mely segítségével a tervezett OO hierarchia (Business Logic) továbbfejleszthető, könnyen felhasználható marad.
- A S.O.L.I.D.-ban leírt szabályok alapján könnyebb az OO hierarchia felépítése is.

S.O.L.I.D. II.

- A S.O.L.I.D. a következő alapelveket tartalmazza:
  - Single-responsibility principle
    - Egyért felelősség alapelve
  - Open-closed principle
    - Nyitottság és zártság alapelve
  - Liskov substitution principle
    - Liskov helyettesítés alapelve
  - Interface segregation principle
    - Interface elkülönítés alapelve
  - Dependency Inversion Principle
    - Függőség invertálás alapelve

S.O.L.I.D. III.

- Az „**Egyért felelősség alapelve**” azt mondja ki, hogy egy osztály, egyszerre csak pontosan egy feladatért lehet felelős és csak akkor változhat, ha ez a feladat változik.
- Tehát kerülendő a multifunkciós osztályok kialakítása.
- Pl.: Ha egy osztály azért felelős, hogy valamilyen számításokat végezzen adatokon, akkor ugyanaz az osztály ne legyen felelős mondjuk ezen adatok megjelenítéséért is.

S.O.L.I.D. IV.

- A „**Nyitottság és zártság alapelve**” azt mondja ki, hogy az osztályoknak nyitottnak kell lenniük a bővítésre, de zártnak a módosításra.
- Tehát minden osztályt úgy kell felépíteni, hogy bővíteni bármikor lehessen, de lehetőleg ne kelljen a már megírt dolgokon módosítani.
- Pl.: Ha egy osztályban (A) egy alprogram azért felelős, hogy más osztályokból (B, C) létrejött objektumok adataival számoljon, akkor azt úgy kell megvalósítani, hogy ha idő közben más osztályok (D, E, …) objektumaival is kell tudjon számolnia, akkor ahhoz az „A” osztályt ne kelljen módosítani.

S.O.L.I.D. V.

- A „**Liskov helyettesítés alapelve**” azt mondja ki, hogyha S altípusa T-nek, akkor a programon belül bárhol helyettesíthető kell legyen egy az S-ből létrejött objektummal a T-ből létrejött objektum.
- Tehát S bővítheti T tudását, de nem csorbíthatja azt.
- Ez tulajdonképpen az öröklés és polimorfizmus elvének felhasználását kontrollálja, hogy a leszármazott osztályok specializált osztályok, de attól még tudják helyettesíteni az őseiket bárhol.

S.O.L.I.D. VI.

- Az „**Interface elkülönítés alapelve**” azt mondja ki, hogy egy interface ne kényszerítsen rá egy osztályra sem olyan alprogramok implementálását, amelyek az osztály feladata szempontjából nem lényegesek, fölöslegesek.
- Tehát interface-ek írásakor úgy alkossuk meg az interface-einket, hogy azokat egy-egy osztályban való implementáláskor pontosan azt kényszerítsük rá az osztályra ami annak kell. Ne írjunk multifunkciós interface-eket.

S.O.L.I.D. VII.

- Pl.: Ha van egy interface, ami egyszerre kényszeríti rá az osztályra, hogy tudjon HTML és XML formátumban is menteni, de van egy osztály amelyik csak XML-be kell menthető legyen, akkor az előző interface-t ketté kell bontani → HTML-be menthető interface-re és XML-be menthető interface-re, és így az osztály amelyik mindkettőbe menthető mindkettőt implementálja, amelyik csak az egyikbe, az csak az egyiket.

S.O.L.I.D. VIII.

- A „**Függőség invertálás alapelve**” azt mondja ki, hogy a magasabb szintű programrészek nem függhetnek az alacsonyabb szintű programrészektől. Azaz minden programrész az absztrakciót kell kövesse és nem a részleteket.
- Tehát minden osztályt úgy kell megírni, hogy a működéshez, és ne egy konkrét megvalósításhoz ragaszkodjon.

S.O.L.I.D. IX.

- Pl.: Van egy BL rétegű osztály ami adatbázist használ a működéséhez. Ezt az osztályt meg lehet írni úgy, hogy maga az osztály kezeli az adatbázist beépítetten, amely így egyértelműen az adott adatbázis kezelővel lesz csak kompatibilis, ehelyett az alapelv értelmében hozzunk létre egy általános kezelő osztályt, amely az adatbázist kezeli és általánosan hozzáférést biztosít az objektumoknak, így ha változtatni kell a felhasznált adatbázis kezelőn, csak egy másik kezelőt írunk, és a BL rétegű osztályt így nem kell módosítani.

Java kollekciók I.

- A Java kollekciók olyan speciális adatszerkezetek, melyek használatával objektumok memóriában történő tárolása, lekérdezése és manipulálása optimális módon történhet.
- Létrejöttükben szerepet játszott, hogy az egyszerű tömbök és mátrixok alkalmatlanok egyes fontos feladatok elvégzésére, illetve a memóriahatékonyságuk is gyenge.
- Használatuk főként OO programozás közben elengedhetetlen, mivel itt már nagyon nagy szerepet játszik a memória kihasználtság optimalizálása, illetve egyes objektumokkal való műveletek csökkentése.

Java kollekciók II.

- Ezek a szerkezetek, általában a rekurzívan bejárható szerkezetek közé tartoznak.
- Ezzel együtt jár, hogy az adatszerkezetek mérete mindössze a memóriától függ, és az is, hogy nem a szokványos tömb felépítését alkalmazzák (teljesen).
- OO környezetben ezen adatszerkezetek megvalósítása természetesen OO alapelveken nyugszik legtöbbször, így osztályok és interfészek testesítik meg az egyes adatszerkezeteket.

Java kollekciók III.

- A kollekciók alkalmazásának előnyei:
  - Hatékony programozás
  - Futási idő csökkenése
  - Dinamikus adatkezelés
  - Különböző típusok kezelése
  - Újrefelhasználhatóság

Java kollekciók IV.

- Ezen kollekciók egységes szerkezetben történő Java megvalósítása az úgynevezett gyűjtemény keretrendszer, a Java Collections Framework.
- JCF három fő részből áll:
  - Interfészek → absztrakt adattípusok reprezentációja
  - Implementációk → az interfészek tényleges megvalósítása
  - Algoritmusok/metódusok → hasznos funkciók:
    - keresés; rendezés; összehasonlítás; bejárás; filterezés